

TP Séance 5 - Questions et Solutions

Partie 1 : Observer la protection JSX

► Q1 : Le script s'exécute-t-il ? Pourquoi ? Que fait React avec les strings dans le JSX ?

Solution :

Le script ne s'exécute PAS.

Pourquoi ? React échappe automatiquement toutes les chaînes de caractères (strings) dans le JSX. Le code HTML malveillant est converti en texte brut avant d'être affiché dans le DOM.

Ce que fait React : React utilise un système d'échappement qui transforme les caractères spéciaux comme <, >, &, etc. en entités HTML (ex: < devient <). Ainsi, le navigateur interprète le contenu comme du texte, pas comme du code HTML exécutable.

Test à faire dans Dashboard :

tsx

```
const dangerousName = '<img src=x onerror=alert("HACK")>';
```

```
// Afficher : <p>{dangerousName}</p>
```

```
// Résultat : Le texte s'affiche, pas d'alerte
```

► Q2 : Que se passe-t-il avec dangerouslySetInnerHTML ?

Solution :

Avec dangerouslySetInnerHTML, le script S'EXÉCUTE et l'alerte apparaît.

Code dangereux (NE PAS UTILISER) :

tsx

```
<div dangerouslySetInnerHTML={{ __html: dangerousName }} />
```

Pourquoi c'est dangereux ? dangerouslySetInnerHTML désactive la protection automatique de React. React injecte directement le HTML dans le DOM sans échappement, ce qui permet l'exécution de scripts malveillants (XSS - Cross-Site Scripting).

Règle d'or : Ne JAMAIS utiliser dangerouslySetInnerHTML avec des données provenant d'un utilisateur ou d'une API externe.

Partie 2 : Authentification JWT simulée

➤ Q3 : Ouvrez Network (F12). Faites un GET /projects. Voyez-vous le header Authorization: Bearer ... ?

Solution :

OUI, le header Authorization est présent.

Comment vérifier :

1. Ouvrir les DevTools (F12)
2. Aller dans l'onglet Network
3. Faire une requête GET vers /projects
4. Cliquer sur la requête
5. Regarder dans "Request Headers"

Ce qu'on doit voir :

text

Authorization: Bearer

eyJ1c2VySWQiOiJlxiwiZW1haWwiOiJhZG1pbkBOYXNrZmxvdY5jb20iLCJyb2xlljoiYWRTaW4iLCJleHAiOiQjE3NDM1OTk5OTk5OTI9

Explication : L'intercepteur Axios ajoute automatiquement le token à chaque requête via `api.defaults.headers.common['Authorization'] = 'Bearer ${token}'`.

➤ **Q4 : Pourquoi stocker le token en mémoire (state React) et PAS dans localStorage ?**

Solution :

Le token est stocké en mémoire (state Redux) car localStorage est vulnérable aux attaques XSS (Cross-Site Scripting).

Risques de localStorage :

- Tout script JavaScript sur la page peut lire localStorage
- Si un attaquant injecte un script (via une faille XSS), il peut voler le token
- Le token volé permet d'usurper l'identité de l'utilisateur

Avantages du stockage en mémoire (state) :

- Le token n'est pas persistant (disparaît au rechargement)
- Non accessible par des scripts tiers
- Plus sécurisé pour les applications sensibles

Note : En production, on utilise souvent des cookies sécurisés (HttpOnly, Secure, SameSite) pour stocker les tokens JWT.

Partie 3 : Redux Toolkit vs ancien Reducer

► **Q5 : Comparez authSlice.ts avec votre ancien authReducer.ts. Qu'est-ce qui a changé ?**

Solution :

Caractéristique	Ancien authReducer.ts	Nouveau authSlice.ts
Syntaxe	Switch/case manuel	createSlice() avec reducers
Action types	Définis manuellement (type: 'LOGIN_START')	Générés automatiquement
Immutabilité	Spread operator manuel (...state)	Immer (semble mutable)
Boilerplate	Beaucoup (interfaces, types, switch)	Très peu
Actions creators	À écrire manuellement	Générés automatiquement

Avantages de Redux Toolkit :

1. **Moins de code boilerplate** : createSlice génère automatiquement actions et reducers
2. **Immer intégré** : On peut écrire state.user = ... (semble mutable) mais c'est immuable en coulisse
3. **Actions creators automatiques** : Plus besoin d'écrire { type: 'LOGIN_START' }
4. **TypeScript intégré** : Meilleure inférence des types

Exemple de différence :

Ancien code :

```
ts
case 'LOGIN_SUCCESS':
  return { ...state, user: action.payload, loading: false };
```

Nouveau code (RTK) :

```
ts  
  
loginSuccess(state, action) {  
  state.user = action.payload.user;  
  state.token = action.payload.token;  
  state.loading = false;  
}
```

Partie 4 : React.memo et optimisation des re-renders

► **Q6 : Combien de composants se re-rendent quand on toggle la sidebar ? Lesquels ne DEVRAIENT PAS ?**

Solution :

Composants qui se re-rendent :

- Sidebar (normal, car isOpen change)
- Dashboard (normal, car son état sidebarOpen change)

Composants qui ne DEVRAIENT PAS se re-render :

- MainContent (ses props columns ne changent pas)
- ProjectForm (non affiché)
- Header (ses props ne changent pas)

Observation avec console.log :

```
tsx  
  
// Dans MainContent.tsx  
console.log('MainContent re-render');  
  
// Ce log apparaît à chaque toggle sidebar (problème d'optimisation)
```

Solution : Utiliser React.memo pour éviter les re-renders inutiles.

► **Q7 : Pourquoi MainContent ne se re-rend plus ? Que compare React.memo ?**

Solution :

Pourquoi ? React.memo est un Higher-Order Component (HOC) qui mémorise le résultat du rendu d'un composant.

Comment ça fonctionne :

tsx

```
import { memo } from 'react';  
export default memo(MainContent);
```

Ce que compare React.memo :

- Il effectue une **comparaison superficielle (shallow comparison)** des props
- Si les props n'ont pas changé (référence identique), le composant n'est pas re-rendu
- Si les props ont changé, le composant est re-rendu

Pourquoi MainContent ne se re-rend plus :

- Les props columns n'ont pas changé (même tableau, même référence)
- React.memo détecte l'absence de changement et skip le re-render

Limitation : Si on passe des fonctions en props, une nouvelle fonction est créée à chaque render, ce qui casse la mémorisation.

► Q8 : Quelle différence entre useMemo et useCallback ? Quand utiliser chacun ?

Solution :

Caractéristique	useMemo	useCallback
Ce qu'il retourne	Une valeur mémorisée	Une fonction mémorisée
Syntaxe	useMemo(() => valeur, [deps])	useCallback(fn, [deps])
Utilisation typique	Calculs coûteux, objets dérivés	Callbacks passés aux enfants memoized
Équivalent	useMemo(() => fn, deps) équivaut à useCallback(fn, deps)	

Quand utiliser useMemo :

tsx

```
// Calcul coûteux à mémoriser
```

```
const expensiveValue = useMemo(() => {  
  return projects.filter(p => p.color === '#1B8C3E');  
}, [projects]);
```

// Objet à mémoriser pour éviter re-render

```
const config = useMemo(() => ({ apiUrl, timeout }), [apiUrl, timeout]);
```

Quand utiliser useCallback :

tsx

// Fonction passée à un composant enfant memoized

```
const handleRename = useCallback((project: Project) => {  
  renameProject(project);  
}, [renameProject]); // La fonction change seulement si renameProject change
```

// Dans le JSX

```
<MemoizedSidebar onRename={handleRename} />
```

Règle d'or :

- useCallback pour les **fonctions** (surtout si passées à des composants memo)
- useMemo pour les **valeurs** (calculs coûteux, objets, tableaux)

Partie 5 : Custom Hook useProjects

► Q9 : (Question ouverte) Quel est l'avantage d'extraire la logique dans un custom hook ?

Solution :

Avantages des custom hooks :

1. **Réutilisabilité** : La même logique peut être utilisée dans plusieurs composants
2. **Séparation des préoccupations** : La logique métier est séparée de la logique d'affichage
3. **Testabilité** : Plus facile à tester de manière isolée
4. **Lisibilité** : Le composant Dashboard devient plus simple et plus clair
5. **Maintenabilité** : Les modifications de la logique CRUD sont centralisées

Exemple de simplification :

Avant (Dashboard.tsx) : ~150 lignes avec toute la logique CRUD

Après (Dashboard.tsx) : ~50 lignes

tsx

```
const { projects, columns, loading, error, addProject, renameProject, deleteProject } =  
useProjects();
```

Le custom hook useProjects encapsule :

- États (projects, columns, loading, error)
- Effets (fetch initial)
- Fonctions CRUD (addProject, renameProject, deleteProject)

Partie 6 : React Profiler

► **Q10 : Pour chaque action, notez : quels composants se re-rendent ? Combien de temps prend le render ? Y a-t-il des re-renders inutiles après vos optimisations ?**

Solution :

Actions à tester avec React DevTools Profiler :

a) Toggle sidebar (ouverture/fermeture)

Composant	Avant optimisation	Après optimisation
Sidebar	✓ Re-render	✓ Re-render (normal)
MainContent	✓ Re-render (inutile)	✗ Pas de re-render
Header	✓ Re-render (inutile)	✗ Pas de re-render
Dashboard	✓ Re-render	✓ Re-render
Temps total	~15-20ms	~5-10ms

b) Ajouter un projet

Composant	Statut
Dashboard	Re-rend (projets modifiés)
Sidebar	Re-rend (liste projets change)
MainContent	Pas de re-rend (colonnes inchangées)
ProjectForm	Se ferme

c) Naviguer vers ProjectDetail

Composant	Statut
Dashboard	Démonte
ProjectDetail	Monte et se rend
Header	Re-rend

d) Se déconnecter

Composant	Statut
Dashboard	Démonte
Login	Monte

Optimisations appliquées :

1. React.memo sur MainContent et Sidebar
2. useCallback sur les fonctions passées en props
3. Custom hook useProjects pour centraliser la logique

Avant/Après avec Profiler :

- **Avant** : Toggle sidebar → 4 composants re-rendus
- **Après** : Toggle sidebar → 2 composants re-rendus uniquement
- **Gain** : ~50% de re-renders en moins

Tableau récapitulatif des optimisations

Optimisation	Problème résolu	Syntaxe
React.memo	Re-renders inutiles des enfants	export default memo(Component)
useCallback	Nouvelles fonctions à chaque render	const fn = useCallback(() => {}, [deps])
useMemo	Recréation d'objets/calculs	const val = useMemo(() => {}, [deps])
Custom Hooks	Logique dupliquée	function useCustomHook() { ... }